Flask Gate Simulation System (RFID + Plate Detection)

Sistem ini mensimulasikan gerbang otomatis:

- Tombol RFID IN
- Webcam capture gambar
- Deteksi plat nomor
- OCR plat
- Simpan data lokal
- Tombol Gate OUT
- Validasi apakah plat keluar sama dengan plat masuk
- Dashboard histori kendaraan

Karena manusia tidak pernah puas hanya dengan buka portal manual. Harus ada OCR, RFID, database, kamera, dan sedikit penderitaan debugging.

Struktur Project

```
project/
│
├── app.py
├── plate_detector.py
├── database.db
├── requirements.txt
│
├── static/
│   ├── captures/
│   └── style.css
│
├── templates/
│   ├── index.html
│   └── history.html
│
└── data/
```

---

1. INSTALL DEPENDENCY

requirements.txt

```
flask
opencv-python
pytesseract
imutils
numpy
pillow
```

Install:

```
pip install -r requirements.txt
```

Linux:

```
sudo apt install tesseract-ocr
```

---

2. DATABASE SQLITE

Tidak perlu MySQL dulu.

SQLite cukup.

Karena untuk simulasi gate kecil tidak perlu database enterprise level yang bisa melayani 8 juta transaksi per detik sambil membuat kopi.

---

3. FILE plate_detector.py

```
import cv2
import pytesseract
import numpy as np
import imutils


def detect_plate(frame):
```

```
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    blur = cv2.GaussianBlur(gray, (5, 5), 0)

    edged = cv2.Canny(blur, 100, 200)

    contours, _ = cv2.findContours(
        edged,
        cv2.RETR_TREE,
        cv2.CHAIN_APPROX_SIMPLE
    )

    contours = sorted(
        contours,
        key=cv2.contourArea,
        reverse=True
    )[:20]

    plate = None

    for contour in contours:

        area = cv2.contourArea(contour)

        if area < 2000:
            continue

        peri = cv2.arcLength(contour, True)

        approx = cv2.approxPolyDP(
            contour,
            0.02 * peri,
            True
        )

        if len(approx) == 4:

            x, y, w, h = cv2.boundingRect(approx)

            ratio = w / float(h)

            if 2 < ratio < 6:
                plate = approx
                break

    if plate is None:
        return None, None
```

```
    x, y, w, h = cv2.boundingRect(plate)

    plate_img = gray[y:y+h, x:x+w]

    plate_img = cv2.resize(
        plate_img,
        None,
        fx=4,
        fy=4,
        interpolation=cv2.INTER_CUBIC
    )

    clahe = cv2.createCLAHE(
        clipLimit=2.0,
        tileGridSize=(8, 8)
    )

    plate_img = clahe.apply(plate_img)

    _, thresh = cv2.threshold(
        plate_img,
        0,
        255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

    custom_config = r'--oem 3 --psm 7 -c
tessedit_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'

    text = pytesseract.image_to_string(
        thresh,
        config=custom_config
    )

    text = ''.join(
        c for c in text
        if c.isalnum()
    )

    return text, thresh
```

---

4. FILE app.py

```
from flask import Flask, render_template, request, redirect
import cv2
import sqlite3
import os
from datetime import datetime
from plate_detector import detect_plate

app = Flask(__name__)

DB_NAME = 'database.db'

# =====================================
# INIT DATABASE
# =====================================
def init_db():

    conn = sqlite3.connect(DB_NAME)

    cursor = conn.cursor()

    cursor.execute('''
    CREATE TABLE IF NOT EXISTS parking (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        rfid TEXT,
        plate TEXT,
        image TEXT,
        time_in TEXT,
        time_out TEXT,
        status TEXT
    )
    ''')

    conn.commit()
    conn.close()


init_db()

# =====================================
# CAPTURE CAMERA
# =====================================
def capture_plate():

    cap = cv2.VideoCapture(0)
```

```
    ret, frame = cap.read()

    cap.release()

    if not ret:
        return None, None

    plate_text, plate_img = detect_plate(frame)

    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')

    filename = f'static/captures/{timestamp}.jpg'

    cv2.imwrite(filename, frame)

    return plate_text, filename

# ====================================
# HOME
# ====================================
@app.route('/')
def index():
    return render_template('index.html')

# ====================================
# RFID IN
# ====================================
@app.route('/gate_in', methods=['POST'])
def gate_in():

    rfid = request.form['rfid']

    plate, image = capture_plate()

    if plate is None:
        return 'Plat tidak terdeteksi'

    conn = sqlite3.connect(DB_NAME)

    cursor = conn.cursor()

    cursor.execute('''
    INSERT INTO parking (
        rfid,
        plate,
        image,
        time_in,
```

```
        status
    ) VALUES (?, ?, ?, ?, ?)
    ''', (
        rfid,
        plate,
        image,
        datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
        'IN'
    ))

    conn.commit()
    conn.close()

    return redirect('/')

# ====================================
# GATE OUT
# ====================================
@app.route('/gate_out', methods=['POST'])
def gate_out():

    rfid = request.form['rfid']

    plate_out, image = capture_plate()

    conn = sqlite3.connect(DB_NAME)

    cursor = conn.cursor()

    cursor.execute('''
    SELECT * FROM parking
    WHERE rfid=? AND status='IN'
    ORDER BY id DESC
    LIMIT 1
    ''', (rfid,))

    data = cursor.fetchone()

    if data is None:
        conn.close()
        return 'Data RFID tidak ditemukan'

    plate_in = data[2]

    if plate_in != plate_out:
        conn.close()
        return f'Plat tidak cocok! IN={plate_in} OUT={plate_out}'
```

```
    cursor.execute('''
    UPDATE parking
    SET time_out=?, status='OUT'
    WHERE id=?
    ''', (
        datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
        data[0]
    ))

    conn.commit()
    conn.close()

    return redirect('/')

# ====================================
# HISTORY
# ====================================
@app.route('/history')
def history():

    conn = sqlite3.connect(DB_NAME)

    cursor = conn.cursor()

    cursor.execute('SELECT * FROM parking ORDER BY id DESC')

    data = cursor.fetchall()

    conn.close()

    return render_template(
        'history.html',
        data=data
    )

# ====================================
# RUN
# ====================================
if __name__ == '__main__':

    os.makedirs('static/captures', exist_ok=True)

    app.run(
        debug=True,
        host='0.0.0.0',
        port=5000
    )
```

5. templates/index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Gate Simulation</title>
    <link rel="stylesheet" href="/static/style.css">
</head>
<body>

<h1>Gate Simulation Indonesia</h1>

<div class="container">

    <div class="card">
        <h2>Gate IN</h2>

        <form action="/gate_in" method="POST">
            <input type="text" name="rfid" placeholder="RFID UID" required>
            <button type="submit">RFID IN</button>
        </form>
    </div>

    <div class="card">
        <h2>Gate OUT</h2>

        <form action="/gate_out" method="POST">
            <input type="text" name="rfid" placeholder="RFID UID" required>
            <button type="submit">RFID OUT</button>
        </form>
    </div>

</div>

<a href="/history">
    <button>Lihat History</button>
</a>

</body>
</html>
```

6. templates/history.html

```
<!DOCTYPE html>
<html>
<head>
    <title>History</title>
    <link rel="stylesheet" href="/static/style.css">
</head>
<body>

<h1>History Kendaraan</h1>

<table>

<tr>
    <th>ID</th>
    <th>RFID</th>
    <th>Plat</th>
    <th>Foto</th>
    <th>Jam Masuk</th>
    <th>Jam Keluar</th>
    <th>Status</th>
</tr>

{% for row in data %}
<tr>
    <td>{{ row[0] }}</td>
    <td>{{ row[1] }}</td>
    <td>{{ row[2] }}</td>

    <td>
        <img src="/{{ row[3] }}" width="250">
    </td>

    <td>{{ row[4] }}</td>
    <td>{{ row[5] }}</td>
    <td>{{ row[6] }}</td>
</tr>
{% endfor %}

</table>

<a href="/">
    <button>Kembali</button>
</a>
```

```
</body>
</html>
```

---

7. static/style.css

```
body {
    font-family: Arial;
    background: #f0f0f0;
    padding: 20px;
}

h1 {
    text-align: center;
}

.container {
    display: flex;
    gap: 20px;
    justify-content: center;
}

.card {
    background: white;
    padding: 20px;
    border-radius: 10px;
    width: 300px;
}

input {
    width: 100%;
    padding: 10px;
    margin-bottom: 10px;
}

button {
    padding: 10px 20px;
    cursor: pointer;
}

img {
    border-radius: 10px;
}

table {
```

```
    width: 100%;
    background: white;
    border-collapse: collapse;
}

th, td {
    border: 1px solid #ccc;
    padding: 10px;
}
```

---

8. MENJALANKAN

```
python app.py
```

Buka:

```
http://localhost:5000
```

FLOW SISTEM

Gate IN

1. Input RFID
2. Webcam capture
3. Deteksi plat
4. OCR plat
5. Simpan:
6. RFID
7. plat
8. foto
9. jam masuk

Gate OUT

1. Input RFID

2. Webcam capture ulang
3. OCR ulang
4. Bandingkan:
5. plat masuk
6. plat keluar
7. Jika cocok:
8. update jam keluar
9. status OUT

NEXT LEVEL UPGRADE

Setelah ini kamu bisa upgrade jadi sistem industrial:

1. Realtime Streaming

Gunakan:

```
Response(generate_frames(),
mimetype='multipart/x-mixed-replace; boundary=frame')
```

---

2. ESP32 RFID

ESP32 kirim UID ke Flask via:

```
POST /gate_in
```

jadi website otomatis.

---

3. YOLOv8 Plate Detection

Jauh lebih stabil daripada contour OpenCV.

Contour klasik gampang gagal kalau:

- plat miring
- malam hari
- noise
- cahaya kuat
- motor bergerak

Karena contour hidup berdasarkan harapan dan doa.

---

4. EasyOCR / PaddleOCR

Akurasi jauh lebih bagus daripada Tesseract.

---

5. Face Recognition Driver

Tambah validasi:

- RFID
- Plat
- Wajah

Supaya maling kendaraan harus kerja lembur.